

Clientside - JavaScript

Korte svar og refleksioner:

Filupload og download i Feedr

1 - Valg af server og API

Hvilken løsning bruger vi?

Jeg har bygget videre på Feedr i ASP.NET Core og .NET 10. Serveren har en Blazor-GUI og et indbygget API. Klienten består af HTML, CSS og JavaScript med jQuery.

Der er ingen login funktion i filløsningen. Databasesiden fra den tidligere opgave er stadig med.

I koden: Program.cs og Feedr.csproj. Alle filhenvisninger i dokumentet er relative til Feedr-mappen.

Hvorfor er det både en GUI-app og et Web API?

Blazor viser serverens sider, mens FilesController tager imod HTTP-kald fra klienten. Begge dele kører i samme app. AddControllers og MapControllers gør API'et tilgængeligt.

MapRazorComponents gør Blazor-siderne tilgængelige.

I koden: Program.cs. Components/App.razor og Components/Routes.razor.

Hvad er forskellen på controllers og Minimal API?

En controller samler endpoints i en klasse, der arver fra ControllerBase. Jeg bruger [HttpGet] og [HttpPost] til at vælge HTTP-metoden. Med Minimal API kan et endpoint i stedet oprettes med fx app.MapGet. MapGet registrerer et endpoint; det sender ikke et kald fra klienten.

Jeg bruger controllers, så filernes endpoints ligger samlet i FilesController.

I koden: Controllers/FilesController.cs.

Kilde: [Microsoft - Web API](#)

Hvorfor Swagger UI frem for Postman?

Swagger UI giver en oversigt og mulighed for at afprøve API'et i browseren. Det er praktisk ved fremvisningen. OpenAPI beskriver endpoints, og Swagger UI fra Swashbuckle viser dem. Postman kan også bruges, hvis man vil gemme og gentage sine requests.

Swagger findes på /swagger, når appen kører i Development.

I koden: Program.cs: AddOpenApi, MapOpenApi og UseSwaggerUI.

2.1 - Server og filer

Hvilke endpoints har vi?

GET /api/files henter filoversigten som JSON.

POST /api/files modtager en fil som multipart/form-data og gemmer den.

GET /api/files/{id}/download henter den valgte fil.

En vellykket upload svarer med 201. Ugyldigt input giver normalt 400, og en fil, der ikke findes, giver 404. For store requests kan også blive afvist med 413.

I koden: Controllers/FilesController.cs: List, Upload og Download.

Hvor bliver filerne gemt?

Filerne gemmes i App_Data/Uploads under serverprojektet. Mappen oprettes automatisk. Filernes indhold gemmes ikke i SQL. Hver upload får et GUID foran sit navn, så to filer med samme oprindelige navn ikke overskriver hinanden.

Under upload bruges en midlertidig fil. Først når skrivningen er færdig, får den sit endelige navn og kommer med i oversigten. API'et viser det oprindelige filnavn sammen med ID, størrelse og tidspunkt.

I koden: Services/FileService.cs: SaveAsync og Describe. Models/StoredFile.cs. appsettings.json: FileStorage.Path.

Hvor bruger jeg LINQ?

ListFiles læser mappens filer med EnumerateFiles. Where vælger de færdige uploads, OrderByDescending sorterer dem efter tidspunkt, og Select laver oplysningerne til oversigten. Den seneste upload står øverst.

I koden: Services/FileService.cs: ListFiles.

Hvordan virker serverens GUI og sletning?

Siden /files viser filerne i en tabel. Slet viser først Bekræft sletning og Fortryd. Ved bekræftelse kalder Blazor FileService.Delete og opdaterer tabellen. Der bruges ikke et ekstra DELETE-endpoint.

Hvis en fil uploades fra et andet vindue, bruges Opdater oversigt. En tom uploadmappe vises som 0 filer med et link til klienten.

I koden: Components/Pages/Files.razor: LoadFiles og DeleteFile. Services/FileService.cs: Delete.

Hvad bruges SQL-databasen til?

SQL bruges stadig af den tidligere databaseside. Ved opstart opretter DatabaseInitializer databasen og indsætter testdata, hvis den er tom. Derfor skal SQL stadig være sat op for at starte hele Feedr-projektet.

I koden: Program.cs og DBAccess/DatabaseInitializer.cs.

2.2 - Klienten

Hvordan er klienten bygget?

Hele klienten ligger i `wwwroot/client`. `index.html` indeholder formular, tabel og en skabelon til rækkerne. `style.css` styrer udseendet, `app.js` indeholder funktionerne, og `config.js` indeholder API-adressen. jQuery ligger lokalt i `lib`.

Klienten bruger ingen Blazor-komponenter eller C#-kode. Dens CSS og scripts indlæses med relative filstier.

Hvor bruger jeg jQuery-events og selectors?

`#upload-form` vælger formularen, og `.on('submit', uploadFile)` håndterer indsendelse. `event.preventDefault` forhindrer en normal sidegenindlæsning. Opdater-knappen bruger et click-event.

Download-eventet er sat på `#file-list` og gælder knapper med klassen `.download-button`. Det virker også på nye rækker. `.find()` vælger felterne i hver række, mens `.text()` sætter deres tekst.

I koden: `wwwroot/client/app.js`: event handlers nederst og `renderFiles`.

Hvorfor jQuery AJAX og ikke fetch?

AJAX er en teknik til asynkron kommunikation uden en fuld genindlæsning. `$.ajax` er jQuerys funktion til det, mens `fetch` er indbygget i browseren. Begge kan løse opgaven.

Jeg bruger `$.ajax`, fordi jQuery allerede kræves til events og DOM. Med `fetch` skulle jeg selv kontrollere `response.ok` ved HTTP-fejl og læse svaret med fx `response.json()`. `$.ajax` kan læse JSON automatisk og afviser sit resultat ved HTTP-fejl.

Kilder: [jQuery AJAX](#) og [MDN Fetch](#)

Hvordan uploades en fil?

`uploadFile` kontrollerer filendelse og størrelse. Filen lægges i `FormData` og sendes med POST. `processData: false` bevarer filindholdet, og `contentType: false` lader browseren sætte den rigtige multipart-header. Efter upload hentes den nye filliste.

I koden: `wwwroot/client/app.js`: `uploadFile` og `loadFiles`.

Hvordan downloades en fil?

`downloadFile` sender GET og modtager en Blob. Det bevarer bytes i både TXT, PDF, DOCX og CSV. Et midlertidigt download-link gemmer filen under dens oprindelige navn. Derefter fjernes linket, og Blob-adressen frigives.

I koden: `wwwroot/client/app.js`: `downloadFile`. `Controllers/FilesController.cs`: `Download`.

Hvor er løsningen asynkron?

Klientens kald bruger `async/await` sammen med `$.ajax`. Serveren skriver uploaden med `CopyToAsync` og streamer downloadet fra en `FileStream`. At afvente et kald med `await` blokerer ikke browserens brugerflade og starter ikke i sig selv en ny tråd.

2.3 - Performance og sikkerhed

Hvordan begrænser jeg arbejdet i browseren?

renderFiles bygger rækker fra den samme HTML-template i et DocumentFragment. Derefter opdateres tabelindholdet samlet. Resten af siden bliver stående. Funktionerne genbruges ved opstart, opdatering og upload.

Listen hentes ved opstart, efter upload og ved klik på Opdater oversigt. Der er ingen timer, der hele tiden spørger serveren. Knapper deaktiveres under relevante kald for at undgå gentagne handlinger.

I koden: wwwroot/client/index.html: file-row. wwwroot/client/app.js: renderFiles, loadFiles, uploading og loading.

Hvorfor validerer både klient og server?

Klienten giver hurtig besked om en forkert filendelse eller størrelse. Klientkoden kan omgås, så serveren kontrollerer også input. Den accepterer .txt, .pdf, .docx og .csv, afviser tomme filer og har en grænse på 10 MB. Filnavne kontrolleres også.

Filtypen kontrolleres ud fra filendelsen. Løsningen indeholder ikke virusscanning eller fuld kontrol af dokumentets indhold.

I koden: Services/FileService.cs: SaveAsync. Controllers/FilesController.cs: RequestSizeLimit og RequestFormLimits. wwwroot/client/app.js: uploadFile.

Hvordan undgår jeg XSS og adgang til andre filer?

Filnavne sættes ind med .text(), så de vises som tekst og ikke udføres som HTML. Blazor koder også filnavnene som tekst. Download bruger et GUID i stedet for en filsti fra brugeren.

Uploadmappen ligger uden for wwwroot. Filerne udleveres som downloads gennem API'et. Filnavne med fx skråstreger afvises. Løsningen udfører ikke SQL med uploadens indhold.

I koden: wwwroot/client/app.js: renderFiles. Services/FileService.cs: SaveAsync og Find. Controllers/FilesController.cs: Download.

Hvordan vises fejl?

Klienten viser en besked tæt på formularen og genaktiverer knapperne efter kaldet. Ved download af en slettet fil får brugeren besked om at opdatere oversigten. På serverens GUI vises en fejl, hvis filer ikke kan læses eller slettes.

I koden: wwwroot/client/app.js: showStatus, catch og finally. Components/Pages/Files.razor: LoadFiles og DeleteFile.

2.4 - Refleksion

Hvad er forskellen på JavaScript og C#?

JavaScript understøtter også objektorienteret programmering. Forskellen er derfor ikke, at JavaScript mangler objekter. JavaScript er dynamisk typet og bygger arv på prototyper. C# er som udgangspunkt statisk typet og bruger klasser og interfaces.

I vores løsning kører JavaScript i browseren og håndterer events og DOM. C# kører på serveren og håndterer API, filsystem og Blazor. Begge sprog har funktioner eller metoder, objekter og async/await.

Kilde: [MDN - JavaScript](#)

Hvor bruger jeg variabler, funktioner og objekter?

files er et array af filobjekter med fx id, name og size. renderFiles bruger en for...of-løkke til at vise dem. if-betingelser kontrollerer blandt andet filstørrelsen. formatSize modtager bytes som parameter, og downloadFile modtager både filobjekt og knap.

I koden: `wwwroot/client/app.js`: `files`, `renderFiles`, `formatSize`, `uploadFile` og `downloadFile`.

Hvad er DOM, render-tree og delvis rendering?

DOM er browserens struktur af HTML-elementer. Browseren bruger DOM og CSS til at danne sit render-tree, beregne placeringer og tegne siden. Når renderFiles ændrer tabelrækkerne, kan browseren opdatere visningen uden at hente hele HTML-siden igen.

Blazor har også et internt render-tree til at beskrive sin brugerflade. På serverens filside ændrer DeleteFile listen, hvorefter Blazor finder ændringerne og opdaterer DOM. @key knytter en række til dens fil-ID.

I koden: `wwwroot/client/app.js`: `renderFiles`. `Components/Pages/Files.razor`: `@foreach`, `@key` og `DeleteFile`.

Kilde: [MDN - Browserens rendering](#)

Hvordan kan klienten flyttes til et andet projekt?

Hele `wwwroot/client` kopieres samlet, inklusive `lib`. `apiBaseUrl` i `config.js` ændres til serverens adresse, fx `http://localhost:5120`. Klientens nye adresse sættes i `FileClient.AllowedOrigins` på serveren. Klienten startes gennem en webserver.

CORS tillader browseren at læse API-svar fra de konfigurerede adresser. Det er ikke et login eller en adgangskontrol for brugere. Relativ indlæsning af CSS og scripts gør, at klienten også kan placeres i en undermappe.

I koden: `wwwroot/client/config.js` og `index.html`. `Program.cs`: `AddCors`. `appsettings.json`: `FileClient.AllowedOrigins`.

3 - Test og fremvisning

Hvordan er løsningen testet?

Der er kørt automatiske API- og browsertests den 7. oktober 2026. Testen brugte en separat uploadmappe og SQL-database. Testdatabasen blev fjernet bagefter. Build gav 0 fejl og 0 advarsler.

1. API, OpenAPI og Swagger kunne åbnes uden login.
2. TXT, PDF, DOCX og CSV blev uploadet og hentet med identiske bytes. Danske tegn, store bogstaver i filendelsen og ens filnavne virkede.
3. Tomme filer, forkerte filendelser, ugyldige filnavne og filer over 10 MB blev afvist. En fil på præcis 10 MB blev accepteret. Manglende filer gav 404.
4. CORS tillod den konfigurerede klientadresse og gav ikke tilladelse til en anden adresse.
5. Upload og download gennem browserens jQuery-kode virkede uden sidegenindlæsning.
6. Klienten afviste en forkert filendelse før API-kaldet. Et filnavn med HTML blev vist som tekst uden udførelse.
7. Sletning og Fortryd virkede i Blazor. Download fra en gammel liste viste en fejl, når filen var slettet.
8. Den eksisterende databaseside og navigationen mellem siderne virkede.
9. Klientmappen blev kopieret til en anden server og undermappe. Upload og download virkede efter ændring af config.js og serverens tilladte adresse.
10. Swagger viste et felt til filupload. Klienten blev kontrolleret på computer og mobilbredde.
11. De uploadede filer var stadig tilgængelige efter genstart.

Hvordan kan løsningen fremvises?

1. Start Feedr, vælg Filklient og upload en fil fra docs-mappen.
2. Download filen og åbn den. Vis, at fillisten blev opdateret uden en fuld genindlæsning.
3. Gå til Filer på serveren, opdater oversigten og slet filen. Vis også Fortryd.
4. Vis endpoints i Test API og de tilsvarende metoder i FilesController.
5. Vis klientens konfiguration og forklar, hvordan mappen kan flyttes.

Hvad er afgrænset?

CSV-filer kan uploades og downloades, men de importeres ikke i SQL. Det er opgavens valgfrie ekstraopgave. Serverens filoversigt opdateres ved besøg, sletning og klik på Opdater oversigt; der er ingen løbende overvågning af andre vinduer.

Opsætning og links til dokumenterne står i README i projektets rod.